UNIVERSIDADE FEDERAL DE VIÇOSA

CAMPUS FLORESTAL

Manual de Gestão de Configuração e Mudança

Projeto: Pingo - Uma Aventura na Rede

Florestal - MG

2025

Sumário

1. Objetivo.....3
2. Ferramentas de Controle de Versão.....3
3. Estrutura das branches.....3
4. Processo de Mudança.....4
5. Commits.....4
6. Branches de Juniores.....4
7. Processo de Pull Request (PR).....5
8. Padrões de código.....6

1. Objetivo

Este manual define os procedimentos de gestão de configuração e mudança adotados pela equipe no desenvolvimento do projeto *Pingo – Uma aventura na rede*. O objetivo é garantir organização, rastreabilidade, padronização e qualidade do código, além de facilitar a colaboração entre todos os membros da equipe.

2. Ferramentas de Controle de Versão

- O projeto utiliza o Git como sistema de controle de versão.
- O repositório oficial está hospedado no **GitHub**.
- Todas as alterações devem passar por controle de versão — não é permitido compartilhar código fora do repositório sem sincronização.

3. Estrutura das branches

O projeto adota o modelo de versionamento Git Flow, com as seguintes branches:

1. **main** → contém a versão estável e pronta para produção.
2. **develop** → integra as funcionalidades em desenvolvimento.
3. **feature/** → usada para desenvolver novas funcionalidades a partir da develop.
4. **release/** → utilizada para preparar uma nova versão antes de integrar à main.
5. **hotfix/** → destinada a correções críticas e imediatas diretamente na main.
6. **docs/** → destinada à documentação do projeto.
7. **{nome}-{disciplina}/sprint#{número}** → branches individuais de juniores criadas para cada sprint.

4. Processo de Mudança

- Toda mudança deve iniciar a partir de uma **issue** registrada no GitHub.
- O desenvolvedor cria uma **branch específica** (feature/, hotfix/, junior/, etc.).

- Após o desenvolvimento, deve ser aberto um **Pull Request (PR)** direcionado à branch **develop** (ou **main**, no caso de hotfixes).
- O **PR** deve conter uma descrição clara da mudança e referência à issue correspondente.
- O código será revisado por, no mínimo, um outro membro da equipe antes da aprovação.

5. Commits

- As mensagens devem ser claras, objetivas e seguir o padrão Conventional Commits, incluindo o número da issue no início.
- Exemplos
  - feat #3: adicionar menu principal.
  - fix #7: corrigir bug de login.
  - docs #12: atualizar README com novas instruções.
- **Commits pequenos e frequentes** são preferíveis a grandes alterações únicas.
- O idioma padrão para mensagens de commit é **português**.

6. Branches de Juniores

- Cada júnior terá sua própria branch em cada sprint.
- O padrão de nomenclatura das branches deve seguir o formato

  {nome}-{disciplina}/sprint#{número}
  - **nome:** do júnior responsável;
  - **disciplina:** a disciplina relacionada (ex: BD, POO, QA);
  - **número:** número da sprint ou da issue.
- As regras de uso são:
  1. Criar a branch a partir da **develop** no início da sprint.
  2. Realizar commits apenas na própria branch.
  3. Abrir um PR para a **develop** ao finalizar.
  4. A PR será revisada por um membro sênior.
  5. Após aprovação, o código é integrado à develop e futuramente à main.

7. Processo de Pull Request (PR)

Todo código novo ou correção deve ser integrado ao repositório por meio de **Pull Request (PR)**. Regras gerais:

1. **Criação do PR**
    a. Deve ser aberto da branch do desenvolvedor (ou júnior) para a **develop**.
    b. O título do PR deve seguir o formato:
        i. **feat #<número da issue>: descrição breve**
        ii. **fix #<número da issue>: descrição breve**
        iii. **docs #<número da issue>: descrição breve**
    c. **O corpo do PR deve conter:**
        i. **O que foi implementado:** resumo das alterações.
        ii. **O que mudou:** impacto e arquivos modificados.
        iii. **Referência:** número da issue ou link correspondente.
        iv. **Teste:** como verificar a mudança.
        v. **Observações adicionais:** dependências ou observações relevantes.
        vi. **Vídeo de demonstração (quando aplicável):**
            1. Obrigatório para tarefas de *front-end* ou visuais.
            2. O vídeo deve ser curto, objetivo e mostrar o funcionamento da funcionalidade.
2. **Revisão e Aprovação**
    a. Cada PR deve ter ao menos um revisor designado.
    b. O autor deve marcar o revisor do seu grupo no GitHub.
    c. O revisor analisa o código, faz sugestões e aprova ou solicita alterações.
    d. Após aprovação, o PR pode ser integrado à branch develop.
    e. Branches de juniores só são aceitas após aprovação de um revisor.

8. Padrões de código

- O projeto segue boas práticas de **Programação Orientada a Objetos (POO)**.
- Os nomes de variáveis, classes e métodos devem ser consistentes e padronizados.
- O código deve estar testado e revisado antes do envio do PR.
- O idioma padrão do código e da documentação é **português**.